



UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

COMPUTER VISION

Lab 6

Fundamental matrix estimation

Author:

AmirMahdi Matin
s5884715

Professors:

Professor Noceti Nicoletta
Professor Solari Fabio

December 18, 2024

1 Part I

1.1 8-point algorithm function (version 1)

In order to implement this algorithm, it is essential to initially construct the "A" matrix for the linear homogeneous system. To accomplish this, the `EightPointsAlgorithm(P1, P2)` function is invoked with the corresponding points $P1$ and $P2$. Subsequently, the Singular Value Decomposition ($SVD(A) = [U \ D \ V]$) is employed to produce the unit matrix V and to identify the final column of V as the solution, which is a 9×1 vector. For the F matrix to be constructed, the subsequent step is to restructure the 9×1 matrix into a 3×3 matrix using `reshape( , [3 3])`. The matrix F must have rank 2, and we manually achieve this. Using the $SVD(F) = [U \ D \ V]$, we then set $D[3,3]$ of the matrix D to 0. The F matrix is subsequently recomputed using UDV^T , in which $D[3,3]$ is equal to zero.

1.2 8-point algorithm function (version2)

The normalized eight-point algorithm is employed to improve the accuracy of the fundamental matrix estimation. Initially, the input points are normalized, and the eight-point algorithm from the final section is implemented. The previous algorithm's stages will be executed once more on the normalized corresponding points in the `EightPointsAlgorithmN(P1, P2)`. In order to emphasize the point, $P1$ and $P2$ must be normalized using the `normalise2dpts(P1)` function. Subsequently, the new F is generated by denormalizing the functionality matrix F using the `EightPointsAlgorithm(P1, P2)`.

It is imperative to emphasize that all of the points are presented in a uniform manner. The numerical stability of the solution can be enhanced by normalizing the input points, which is involved in the solution of a linear system of equations in the 8-point algorithm. It aids in the prevention of problems associated with ill-conditioned matrices that may occur when dealing with large or small numbers in the input data. The algorithm becomes less susceptible to anomalies and noise in the data by establishing a common scale for the point coordinates. The fundamental matrix is defined to scale. The algorithm's scale-invariance is guaranteed by normalizing the coordinates. Normalization facilitates the optimization process by facilitating the attainment of more dependable and expedited convergence. It can enhance the solution's robustness, particularly in situations where the point correspondences exhibit a broad spectrum of scales.

1.3 Evaluation of the results

A script titled "Main" has been developed to verify the accuracy of the results and the implemented method, utilizing two distinct sets of stereo images for validation. Thirteen correspondence points have been computed for the Rubik instance to evaluate how these two method variants may estimate the fundamental matrix. The value of $X^T F X$ at each point indicates the accuracy of the estimation of F . This value should ideally equal zero. Nonetheless, in estimating the matrix F , it may be seen as a constraint or an error. The error value for two algorithms, one with normalization and one without, is determined and displayed below. The nearer these values are to zero, the more effectively the fundamental matrix associates the epipolar lines from a place in one image to its corresponding position in the subsequent image:

$$X^T F X = \begin{bmatrix} 0.0905 & 0.7632 & 0.7653 & 0.1886 & 0.0934 & 0.2757 & 0.2190 & 0.6220 & 0.4213 & 0.1686 \\ 0.8453 & 0.0050 & 0.1218 & & & & & & & \end{bmatrix}$$

$$X^T F X_N = \begin{bmatrix} 0.0030 & 0.0012 & 0.0023 & 0.0020 & 0.0014 & 0.0011 & 0.0012 & 0.0013 & 0.0035 & 0.00095 \\ 0.0012 & 0.0015 & 0.0014 & & & & & & & \end{bmatrix}$$

Both matrices are evidently near zero; nevertheless, the matrix with normalized matching points contains elements far smaller than the other, indicating superior estimation accuracy.

The matching spots and the epipolar lines linking them for the two images titled "Rubik" are illustrated for the non-normalized and normalized algorithms, respectively, in Figures 1 and 2:

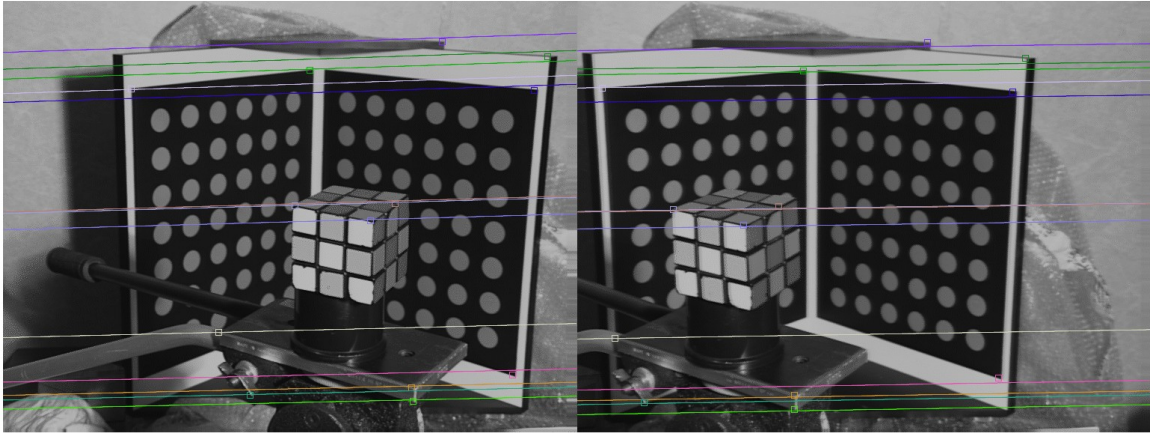


Figure 1: Rubik epipolar lines for the algorithm version1

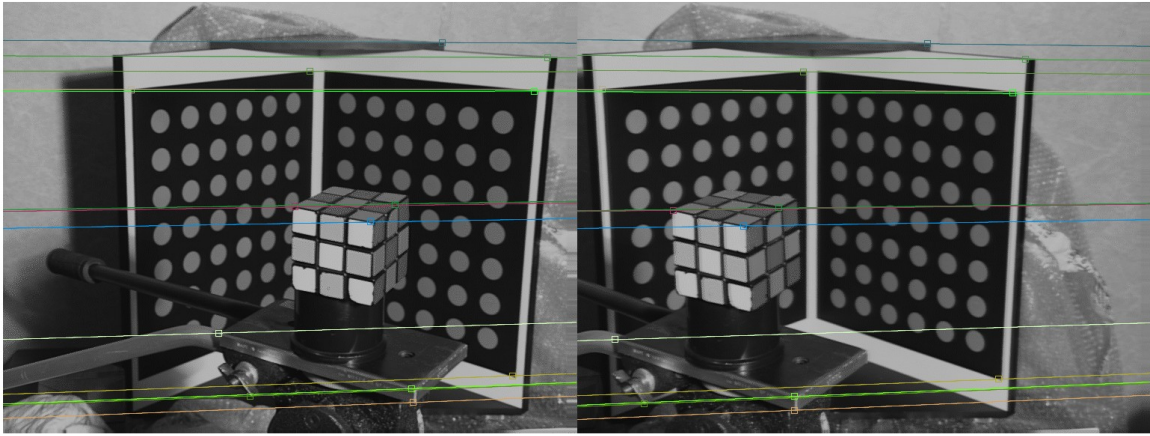


Figure 2: Rubik epipolar lines for the algorithm version2

As a consequence of this, the epipolar lines are depicted with a lower degree of precision in Figure 1, which does not have any normalized corresponding points, in comparison to Figure 2, which contains normalized corresponding points. In this case, the epipolar lines do not cross exactly the points that correspond to them, whereas in normalized ones, the lines pass close to or through the correspondences with a higher degree of accuracy.

All of the procedures described above are performed for another photo that is referred to as "Mire," and the results are displayed in a manner that is crystal clear. In Figure 3, the epipolar lines are depicted using algorithm version 1, and in Figure 4, the normalized matching points are depicted in their respective illustrations.

Taking a look at the constraint equation for this collection of photographs, it is clear that the fundamental matrix for this collection is a more accurate estimation than Rubik's pictures. This is because the result of the error is significantly closer to zero than it is with Rubik's pictures.

It is important to point out that the Main script encompasses the codes that are developed for this particular section. As a result of commenting on the relative names of the photographs and their points and uncommenting the names of other images, it is possible to evaluate the functionality of these two methods on a variety of images.

$$X^T F X = \begin{bmatrix} 0.0072 & 0.0018 & 0.0073 & 0.0100 & 0.0227 & 0.0253 & 0.0252 & 0.0547 & 0.0246 & 0.0401 \\ 0.0427 & 0.00079 & 0.0192 & 0.0115 & 0.0010 & & & & & \end{bmatrix}$$

$$X^T F X_N = \begin{bmatrix} 0.0094 & 0.0004 & 5.858 \times 10^{-06} & 0.0050 & 0.0031 & 0.0097 & 0.0048 & 0.0072 & 0.0030 & 0.0010 \\ 0.0092 & 0.0043 & 0.0021 & 0.0099 & 0.069 & & & & & \end{bmatrix}$$

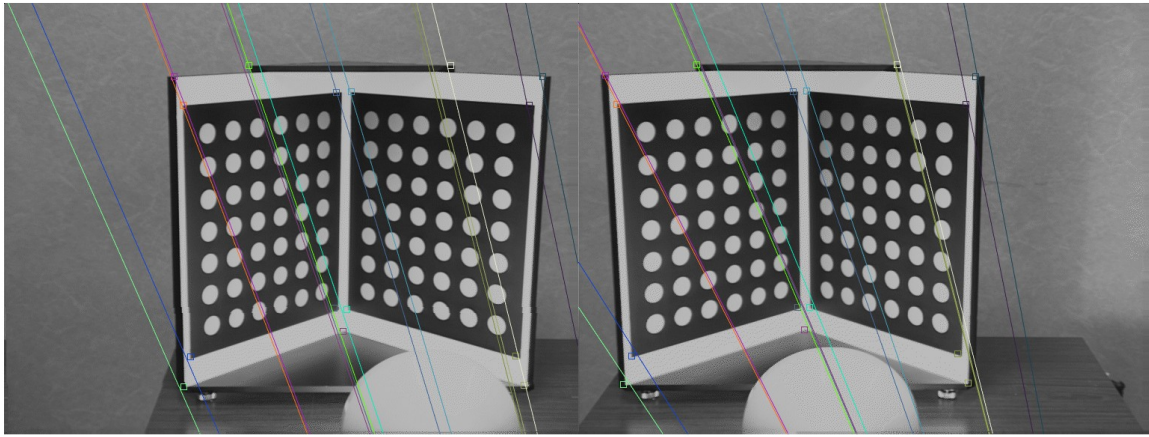


Figure 3: Mire epipolar lines for the algorithm version1

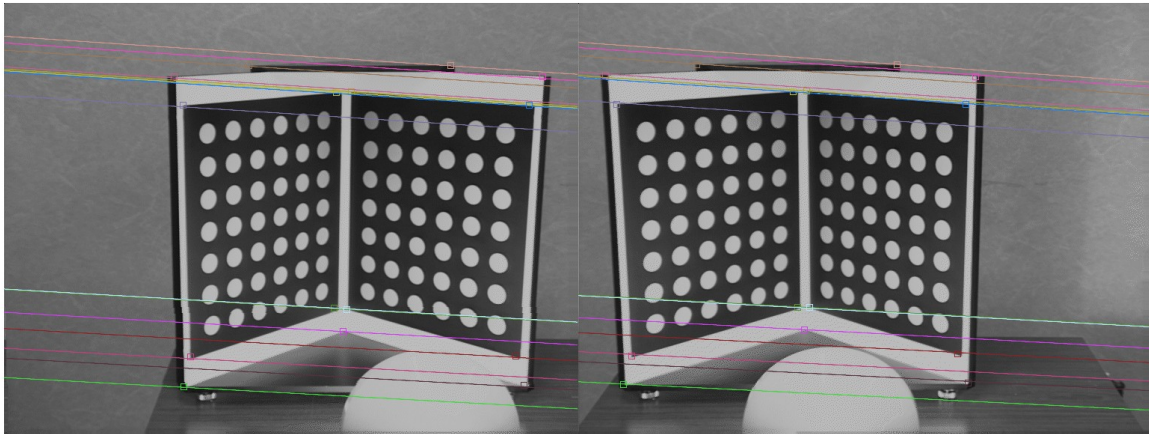


Figure 4: Mire epipolar lines for the algorithm version2

Examining the epipoles reveals the same result as the previous procedure. Computing the left and right epipoles, which are the last columns of U and V in $\text{svd}(F)$, respectively, is the process that is carried out. It is also important that the left and right epipoles are aligned with the epipolar line. As a consequence of this, the epipolar constraint ought to be satisfied by these two epipoles ($e_l^T F e_r = 0$). After computing the value of this constraint for both fundamental matrices, it is found that both of them are extremely small. For example, the value for F_1 is 6.15×10^{-22} , and the value for F_2 is 9.8187×10^{-25} .

2 Part II

In contrast to the first portion, this section does not come with the correspondences already provided. By utilizing the `findMatches` function, the objective is to locate the points that relate to each other first. Because this function could find a great number of points, a significant number of them could be incorrect or noisy. Consequently, the inliers, or the matches that are correct, ought to be communicated. The subsequent phase is analogous to the first step, which is to compute the "A" matrix and the fundamental matrix F , and then to locate epipolar lines together with the points that correspond to them after they have been normalized.

Getting back to the problem of identifying matching points, there are a few different approaches to obtain them, such as the Normalized Cross-Correlation (NCC) and the Scale-Invariant Feature Transform (SIFT). While SIFT will locate a large number of comparable points, it is important to note that not all of these points are inliers. For the purpose of filtering the correspondences, putting the outliers aside, and calculating the optimal fundamental matrix, the RANSAC (RANdom SAMpling Consensus) approach is utilized. This method chooses a few pairs of corresponding points at random from the points that were discovered by the `findMatches` function. These pairs are then used in the `EightPointsAlgorithmN(P1, P2)` function, which locates the fundamental matrix F , computes the $X^T F X$, which can be considered as the error (we considered the absolute value of $X^T F X$ for testing the fundamental matrix), and establishes a threshold. In this procedure, the outliers are identified, the inliers are retained, and the process is repeated for a significant number of repetitions. A change is also made to the total number of iterations. Whenever a superior fundamental matrix F is discovered, the one that was previously used will be substituted for it.

Now, there ought to be proper inlier pair points that appear to be accurate, and the error ought to be very close to zero; hence, the threshold ought to be getting smaller. In the end, one accurate F would be discovered, along with the points that corresponded to it the best. When it was first implemented, the `findMatch` algorithm was utilized in conjunction with two distinct NCC and SIFT approaches. During the simulation of the code, it appeared that the NCC algorithm was unable to locate any matches in the two photos by employing the delta and sigma values of 5 and 10, respectively. Changing the values of these two factors and attempting to discover the appropriate value while taking into consideration the photographs is one approach that may be taken to try to solve this problem. In an effort to reduce the quantity of sigma, which represents the contribution of the Euclidian distance, and delta, which represents the size of the patch for the NCC technique, we made multiple attempts. This is Figure 5, which displays the findings for the scenario in which sigma and delta are equal to 5 and 4, respectively. In spite of the fact that the algorithm has been successful in locating matches using this method, it is immediately apparent that the matches are not accurate in any way, and flaws can be observed just by looking at the photographs.

When it came to the SIFT approach, I had the same issue. This method uses a value for the comparison of the vectors that is equal to 0.4 for this particular example. This value is referred to as the sigma. In addition, the outcomes of this particular example are displayed in Figure 6. For the second time, it is clear that this method does not display correct matches in the same way that the NCC method does. At first glance, the results of NCC appeared to be more plausible than those of SIFT because the error was lower. There were other checks made on other values for the parameters that were given. It was not possible to find appropriate matches in any of the circumstances.

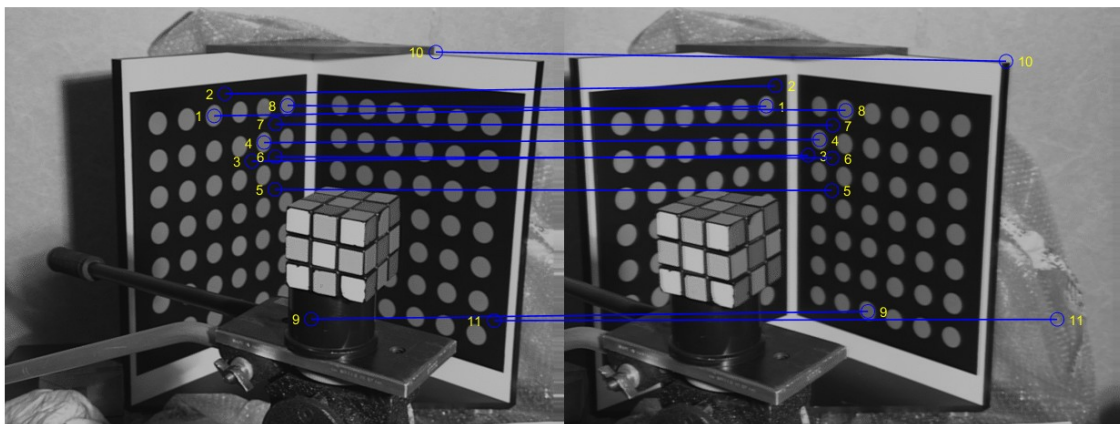


Figure 5: Rubik correspondence points finding algorithm using NCC

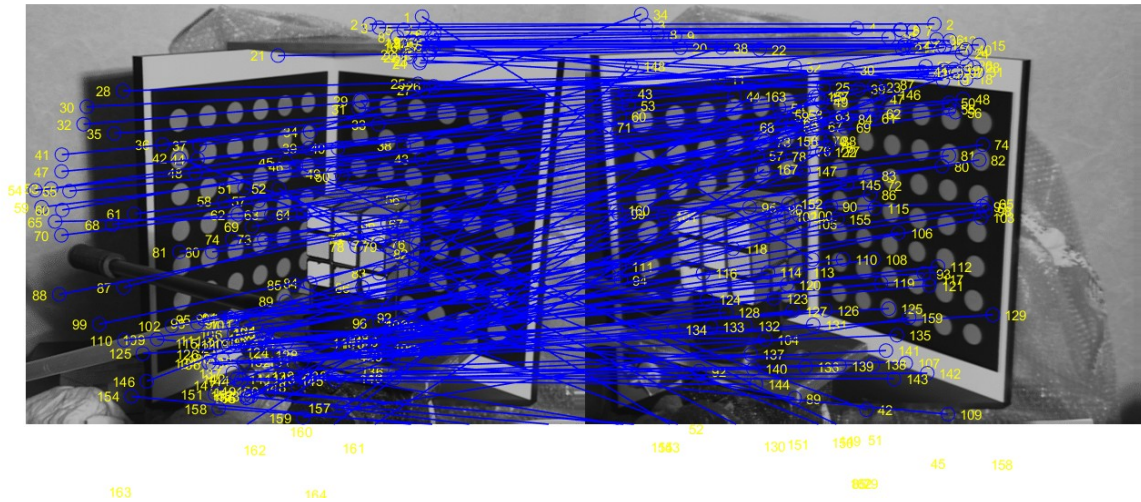


Figure 6: Rubik correspondence points finding algorithm using Sift

The `Ransac` function was finished before moving on to the following step. The `ransac` function received the matches that were discovered in the previous section as inputs. These matches were delivered to the function. There is also a threshold that is defined here, and it is equal to 0.1 for the image that is being represented. Because the `Ransac` function had access to a greater number of correspondence points and the ability to select a variety of random points, it was given the SIFT method's correspondence points. The absolute value of the expression $X^T F X$ would be considered the test function or the error, according to the definition.

Figure 7 demonstrates that the fundamental matrix that has been optimized is not accurate since the epipolar lines are not shown in the correct manner. The reason for this is primarily due to the fact that the `Ransac` function is an optimization method, and if the inputs are incorrect, then the optimization and the optimal fundamental matrices will also be incorrect. In order to function properly, RANSAC is predicated on the premise that the majority of the data points are inliers and that the model should be able to adequately fit these inliers. If a substantial number of incorrect matches are provided, the algorithm may attempt to fit the model to these outliers rather than the actual underlying model, which will result in the generation of an incorrect model because of this. Due to the presence of noise and complexity in the photographs, the majority of the matches that were discovered for this collection of pictures, which can also be seen in Figure 6, are incorrect. As a result of this issue, the `Ransac` function produces incorrect resulting values.

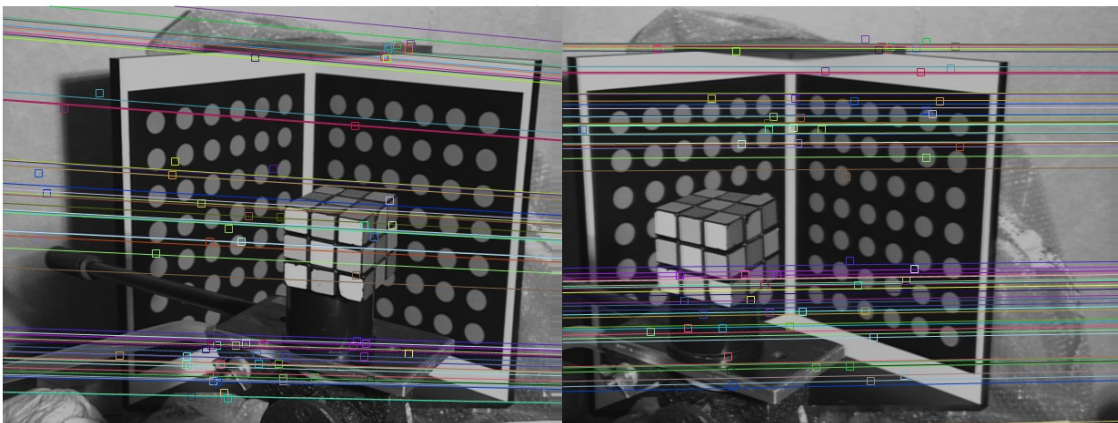


Figure 7: Rubik correspondence points finding algorithm using Sift

It is possible to see that the values of the epipolar constraint expression ($X^T F X$) for the output points of the `Ransac` function are actually very close to zero. This indicates that the `Ransac` function has been successful in locating an optimized fundamental matrix; however, there are some matches that are not recognized in the correct manner. In spite of the fact that the epipolar constraint is tested on both the left and right epipoles, the

result is quite near to zero. In order to lessen the effect that incorrect matches have on RANSAC, it is essential to preprocess and filter the data with great care before applying the algorithm.

It is written in the `Main2` MATLAB script that the primary script for this portion is constructed. For the purpose of observing the results concerning the relevant set of photographs, all that is required is to change or remove the comment from the names of the input images. It should be brought to your attention that the `findMatches` function, which has a significant impact on the outputs, can be used with a variety of values, depending on the image being analyzed. For these arguments, there are also some values that have been tested and commented on within the `findMatches` method.